

▼ Benchmarky a vyhodnocení modelu

V tomto notebooku navazujeme na předchozí zpracování dat. Načteme si rozdělené sady (Train a Validation), aplikujeme identický preprocessing jako v prvním kroku a stanovíme základní výkonnostní metriky pomocí naivního modelu a logistické regrese.

```
import pandas as pd
import numpy as np
import os
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, accuracy_score, classification_report, brier_score_loss

# --- 1. AUTOMATICKÉ STAŽENÍ DAT ---
os.makedirs('data', exist_ok=True)

# ZDE DOPLŇ ID SVÝCH SOUBORŮ (najdeš je v odkazu pro sdílení na Drive)
# Příklad: !gdown --id ID_SOUBORU -O cesta/kam/uložit
print("Stahuji data...")
!gdown --id 13fB4Z9a0L7gM8cNDwCZPvWbKj1V1lljo -O ks-projects-201801.csv # Tvůj hlavní soubor

# --- 2. REPRODUKCE ČIŠTĚNÍ A SPLITU ---
# Aby byl notebook 100% nezávislý, provedeme rychlé vyčištění a split přímo zde
df = pd.read_csv('ks-projects-201801.csv')

# Základní čištění (identické s prvním notebookem)
df_clean = df[df['state'].isin(['successful', 'failed'])].copy()
df_clean['target'] = (df_clean['state'] == 'successful').astype(int)
df_clean = df_clean.drop(columns=['pledged', 'backers', 'usd_pledged', 'usd_pledged_real', 'ID', 'name'], errors='ignore')

# Časový split
df_clean['launched'] = pd.to_datetime(df_clean['launched'])
df_clean['deadline'] = pd.to_datetime(df_clean['deadline'])
df_clean = df_clean.sort_values(by='launched').reset_index(drop=True)

train_idx = int(len(df_clean) * 0.70)
val_idx = int(len(df_clean) * 0.85)

df_train = df_clean.iloc[:train_idx].copy()
df_val = df_clean.iloc[train_idx:val_idx].copy()

# --- 3. PREPROCESSING ---
def add_features(data):
    d = data.copy()
    d['duration_days'] = (d['deadline'] - d['launched']).dt.days
    return d

X_train_raw = add_features(df_train)
X_val_raw = add_features(df_val)

num_cols = ['usd_goal_real', 'duration_days']
cat_cols = ['main_category', 'country']

y_train = X_train_raw['target']
X_train = X_train_raw[num_cols + cat_cols]
y_val = X_val_raw['target']
X_val = X_val_raw[num_cols + cat_cols]

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
    ])

preprocessor.fit(X_train)
X_train_processed = preprocessor.transform(X_train)
X_val_processed = preprocessor.transform(X_val)

print("---- VŠE PŘIPRAVENO K BENCHMARKU ----")
```

```
Stahuji data...
/usr/local/lib/python3.12/dist-packages/gdown/__main__.py:139: FutureWarning: Option `--id` was deprecated in version 4.3.1
  warnings.warn(
Downloading...
From: https://drive.google.com/uc?id=13fB4Z9a0L7gM8cNDwCZPvWbKj1V1lljo
To: /content/ks-projects-201801.csv
100% 58.0M/58.0M [00:00<00:00, 133MB/s]
--- VŠE PŘIPRAVENO K BENCHMARKU ---
```

```
# --- 1. NAIVNÍ BASELINE ---
baseline_preds = np.zeros(len(y_val))
baseline_probs = np.zeros(len(y_val))

# --- 2. LOGISTICKÁ REGRESE ---
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_processed, y_train)

ml_preds = log_reg.predict(X_val_processed)
ml_probs = log_reg.predict_proba(X_val_processed)[: , 1]

# --- 3. VÝSLEDKY ---
results = pd.DataFrame({
    'Metrika': ['Accuracy', 'F1-Score', 'Brier Score'],
    'Naivní Baseline': [
        accuracy_score(y_val, baseline_preds),
        f1_score(y_val, baseline_preds, zero_division=0),
        brier_score_loss(y_val, baseline_probs)
    ],
    'Logistická Regrese': [
        accuracy_score(y_val, ml_preds),
        f1_score(y_val, ml_preds),
        brier_score_loss(y_val, ml_probs)
    ]
})

display(results.round(4))
```

	Metrika	Naivní Baseline	Logistická Regrese
0	Accuracy	0.6259	0.6580
1	F1-Score	0.0000	0.3948
2	Brier Score	0.3741	0.2141

9. Krátké shrnutí

Klíčové metodické body

Na základě zpětné vazby z DÚ1 jsme implementovali postupy pro zajištění maximální robustnosti modelu:

- **Prevence Time Leakage:** Použili jsme striktní chronologický **time-based split** (podle data `launched`). Tím jsme zajistili, že model testujeme na schopnost generalizace v čase a simulujeme reálné nasazení (obrana proti **concept driftu**).
- **Integrita preprocessingu:** Veškeré transformace (škálování, kódování) byly učeny (**fit**) výhradně na trénovacích datech, čímž jsme eliminovali riziko *preprocessing leakage*.
- **Pre-launch focus:** Model pracuje pouze s featurami, které jsou tvůrci známy před ostrým startem kampaně.

Analytické výzvy a insights

Nejobtížnější částí bylo vyladit metriky tak, aby odpovídaly **cost-sensitive** povaze byznys zadání.

- Protože chyba typu **False Positive** (falešný příslib úspěchu) představuje pro tvůrce nejvyšší riziko ztráty investic, rozšířili jsme evaluaci o **Brier Score**.
- Tato metrika nám pomáhá hodnotit kvalitu **kalibrace pravděpodobností** – cílem je vracet tvůrci spolehlivý odhad šance na úspěch, nikoliv jen strohou binární klasifikaci.

Další kroky (Next Steps)

V navazující fázi projektu se zaměříme na:

1. **Pokročilé modely:** Experimentování se složitějšími algoritmy (např. **Random Forest** nebo **XGBoost**).
2. **Rozšířený Feature Engineering:** Vytvoření časových příznaků (např. sezónnost, vliv měsíce spuštění).
3. **Thresholding:** Explicitní práce s posouváním rozhodovacího prahu pro aktivní potlačení drahých False Positive chyb.

